



Carátula para entrega de prácticas

Laboratorio de computación salas A y B

Profesor: M.C Rene Adrian Davila Perez

Asignatura: Programación orientada a objetos

Grupo: 7

*No de
Práctica(s):* 2

Integrante(s): 323036285
323279866
323209432
323161891
323063609
314306908

*No. De Lista o
brigada* 3

Semestre: 2027-1

*Fecha de
entrega:* 4/ 09 / 2026

Obervaciones:

CALIFICACIÓN: _____

Índice

1. Introducción	2
2. Marco Teórico	3
3. Desarrollo	4
4. Resultados	8
5. Conclusiones	9

1. Introducción

Cuando se inicia en un nuevo lenguaje de programación, no se sabe cómo emplear correctamente las estructuras de control condicionales y cíclicas, además, se encuentra dificultad en el uso correcto de variables, causando retrasos con la resolución de problemas.

Al mismo tiempo, se solicita apoyo a los LLM para diseñar distintas aplicaciones, pero las respuestas generadas suelen dificultar la comprensión del código propuesto, generando retrasos en el aprendizaje y problemas de optimización.

Resolver estos problemas nos ayudará a comprender el funcionamiento de los ciclos y a generar tablas, además de que nos dará herramientas para hacer el programa con el que asista el LLM, sus respuestas y optimizar el desarrollo del programa.

Objetivos

- Verificar lo que recomienda un “LLM” acerca del programa “Estudiantes aprobados y Reprobados”, para darse ideas de como realizarlo y comenzar a discernir lo que es útil.
- Elaborar programas donde se usen estructuras cíclicas, para lograr su entendimiento y emplearlas correctamente en futuros programas.

2. Marco Teórico

El método `main()` en Java es el punto de entrada de la JVM (Máquina Virtual de Java) al programa Java. [2]

La palabra clave `static` es un modificador sin acceso que se utiliza para métodos y atributos, permitiendo acceder a ellos sin necesidad de crear un objeto de la clase. [3]

El método estático `main()` indica claramente a la JVM que debe llamarlo para iniciar la aplicación Java. De lo contrario, sería necesario especificar manualmente la función de entrada en cada compilación. Además, es estático porque, de otra manera, existiría ambigüedad sobre qué constructor debe invocarse. [2]

La clase `Scanner` se utiliza para obtener la entrada del usuario y se encuentra en el paquete `java.util`. Para usarla, se crea un objeto de la clase y se emplean sus métodos disponibles; por ejemplo, `nextLine()` para leer cadenas de texto. [4]

Las estructuras de control en Java permiten alterar el orden de ejecución de las instrucciones de un programa. Los bloques de instrucciones se delimitan con corchetes.

1. **Estructura if:** Evalúa una condición y ejecuta un bloque de instrucciones si esta es verdadera.
2. **Estructura for:** Crea un bucle que se ejecuta mientras una condición sea verdadera; el iterador cambia su valor en cada ciclo.
3. **Estructura while:** Mantiene el bucle mientras la condición especificada sea verdadera.
4. **Estructura switch:** Selecciona un bloque de instrucciones según el valor de la sentencia `switch`; si coincide con un caso, se ejecuta el bloque asociado. Incluye opcionalmente `break` y `default`.
5. **Estructura do-while:** Similar a `while`, pero garantiza que el bloque se ejecute al menos una vez, ya que la condición se evalúa después de la primera ejecución. [1]

3. Desarrollo

1. Programa Estudiantes aprobados y Reprobados

Para este punto de la práctica se usaron dos prompts y se analizaron las diferentes respuestas:

■ Prompt utilizado 1

Te voy a mandar un ejercicio el cual nos dio nuestro profesor de POO: Investigar con LLM la generación de la aplicación de control de aprobados y reprobados, con compensación cuando hay cierto número de aprobados. Ese es el ejercicio, te voy a mandar foto de la última salida, y este mismo programa mantiene una condición en la cual todos pasan y se le bonifica el profe; todos pasan menos uno y aún se bonifica; después todos menos dos y no se le bonifica. También se nos mostró que cuando todos reprueban se despliega un mensaje especial. Siempre se indica cuántos aprobaron y cuántos no. El objetivo es diseñar el algoritmo y explicar cómo se solucionaría el problema.

Diseño del algoritmo

- a) **Entrada de datos:** Definir cuántos alumnos se van a evaluar (puede ser fijo o preguntado al usuario). Inicializar dos contadores en cero: **aprobados** y **reprobados**.
- b) **Recolección de resultados (ciclo):** Repetir tantas veces como alumnos haya. Pedir al usuario un valor (1 = aprobó, 2 = reprobó). Si es 1, incrementar **aprobados**. Si es 2, incrementar **reprobados**. Si no es ninguno de los dos, avisar que es inválido y repetir esa entrada.
- c) **Reporte de resultados:** Al terminar el ciclo, mostrar el total de aprobados y reprobados.
- d) **Lógica de compensación:** Evaluar la condición sobre reprobados:
 - Si **aprobados == 0** → todos reprobaron → mensaje especial, sin bonificación.
 - Si **reprobados == 0** → todos aprobaron → se otorga bonificación.
 - Si **reprobados == 1** → aprobaron todos menos uno → se otorga bonificación.
 - Si **reprobados >= 2** → dos o más reprobaron → no se otorga bonificación.

Prompt utilizado 2

Ejercicio 1. Investigar con LLM la generación de la aplicación de control de aprobados y reprobados de 10 alumnos, con compensación cuando hay mínimo 9 aprobados. También avisa cuántos aprobaron y cuántos no. Dame el algoritmo y cómo solucionar el problema.

Conforme a las estructuras de control, tipos de datos primitivos y el uso de la clase utilitaria **Scanner** revisados en las Guías Prácticas 02 y 03 del manual MADO-22 y el Tema 1 del programa de estudio oficial, se presenta el diseño del algoritmo y la solución técnica.

Análisis del Problema

- **Entrada:** Registro secuencial del resultado de 10 alumnos. Para cada alumno se ingresa un indicador binario (1 = Aprobado, 2 = Reprobado).
- **Restricción:** No se registran calificaciones numéricas, únicamente la condición de acreditación.
- **Procesamiento:** Contabilizar el total de aprobados y reprobados mediante un ciclo exacto de 10 iteraciones.
- **Condición de Compensación:** Si la cantidad de alumnos aprobados es mayor o igual a 9 (≥ 9), se despliega un aviso de compensación.
- **Salida:** Total de aprobados, total de reprobados y el mensaje condicional de compensación si se alcanzó el umbral.

Algoritmo

- a) Inicio.
- b) Declarar constantes y variables:
 - Constante `TOTAL_ALUMNOS = 10`.
 - Contadores enteros inicializados en cero: `aprobados = 0`, `reprobados = 0`.
 - Variable entera para la entrada: `resultado`.
 - Instanciar el lector de consola **Scanner**.
- c) Ciclo de captura (desde `i = 1` hasta `TOTAL_ALUMNOS` con incremento de 1):
 - Solicitar al usuario el resultado del alumno actual: Ingrese resultado (1 = Aprobó, 2 = Reprobó).
 - Leer resultado desde la consola con `nextInt()`.
 - Validación/Selección (**if-else**):
 - Si `resultado == 1`: incrementar `aprobados++`.
 - Si `resultado == 2`: incrementar `reprobados++`.
 - De lo contrario: mostrar mensaje de entrada inválida y repetir la captura.
- d) Despliegue de resultados:
 - Imprimir en consola el total de alumnos aprobados.
 - Imprimir en consola el total de alumnos reprobados.
- e) Evaluación de compensación:

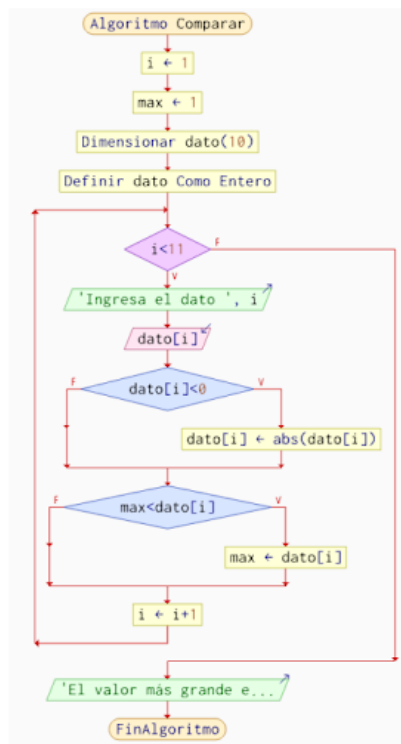
- Evaluar condición: `if (aprobados >= 9)`.
 - Si verdadera: imprimir "Bono / Compensación otorgada".
- f) Liberar recurso cerrando el escáner (`sc.close()`).
- g) Fin.

Al comparar las respuestas, se observó que la principal diferencia entre los modelos fue el formato y la información. Un modelo dio el algoritmo detalladamente paso a paso, usando un enfoque más didáctico, mientras que otro dio un enfoque más técnico. Otra diferencia muy notoria se ve en la forma de hacer el código, ya que mientras un modelo ideó el programa para que el usuario ingresara el número de estudiantes, mientras que otro lo fijó en 10, así como uno a la hora de la salida se basa más en el número de aprobados, otro se centra en los reprobados.

2. Ejercicio Comparar

En el segundo ejercicio se trabajó con estructuras cíclicas para el procesamiento de una serie de datos, los cuales son diez números enteros y de estos se determina el valor máximo entre ellos, manteniendo la restricción para su solución de únicamente tres variables (contador, número y mayor/ más grande).

Se siguió el siguiente algoritmo:



El resultado se centra en el método estático main, un punto de entrada ejecutado directamente por la Máquina Virtual de Java que no requiere la creación de una

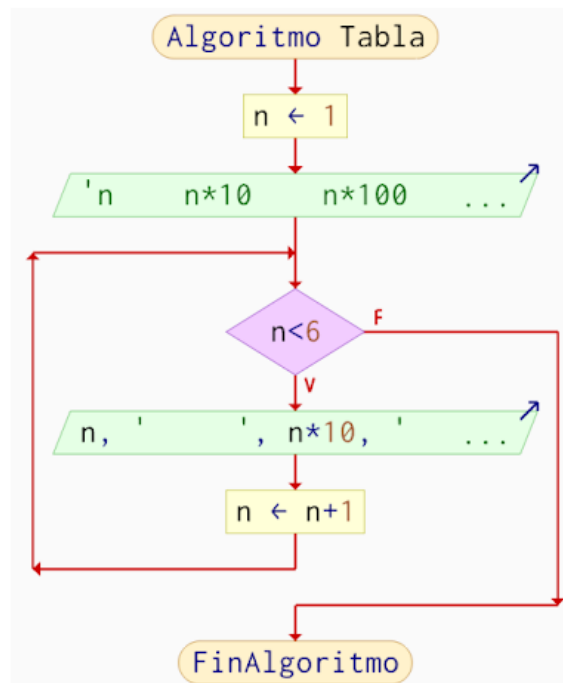
clase previa. A través de la importación de la clase utilitaria Scanner del paquete java.util, se gestiona la captura de datos desde la consola. La lógica de iteración se fundamenta en una estructura while, la cual controla diez lecturas consecutivas mediante una variable de control actualizada con el operador contador++.

Tras esto, se emplean estructuras de selección simple if con operadores el operador num *= -1 para que los valores negativos tengan magnitud positiva y comparar cada entrada con una variable para determinar el valor máximo. Después, la salida en consola utiliza el operador concatenación (+) para convertir los números enteros a cadenas de texto. Finalmente, con la función scan.close() evitamos que datos indeseados sean introducidos.

3. Ejercicio tabla

El tercer ejercicio consiste en la generación de una tabla de valores mediante un ciclo, mostrando la relación entre un número n y sus múltiplos (n, 10n, 100n, 1000n).

Se siguió el siguiente algoritmo:



Al igual que en el ejercicio anterior nos centramos en el método estático main. Declaramos una variable de tipo primitivo entero que actúa como variable de control para una estructura while que delimita cinco iteraciones mediante la expresión (i < 6) y el operador i++. A través del flujo de salida estándar Sys-

tem.out.println, se imprime una tabla de potencias de diez utilizando sangrías horizontales para la alinear las columnas con (/t). Por último, gracias a la jerarquía de operaciones se realiza primero las multiplicaciones entre i y la potencia de 100 y luego con el signo de suma se concatenan los resultados en una sola cadena de texto.

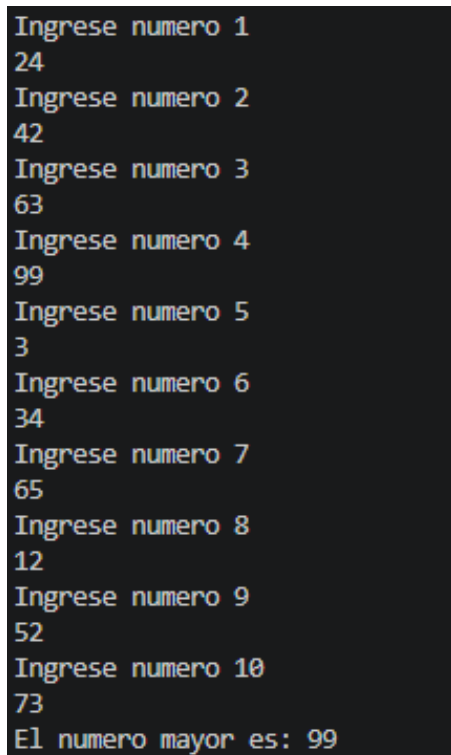
4. Resultados

Tras concluir el desarrollo del código, realizamos las pruebas de validación necesarias para asegurar los resultados.

1. Ejercicio Comparar

Para el ejercicio 2, el programa pide ingresar diez números, uno por uno a través del teclado.

Después de comparar los números muestra en pantalla cuál fue el mayor de toda la secuencia.



```
Ingrese numero 1
24
Ingrese numero 2
42
Ingrese numero 3
63
Ingrese numero 4
99
Ingrese numero 5
3
Ingrese numero 6
34
Ingrese numero 7
65
Ingrese numero 8
12
Ingrese numero 9
52
Ingrese numero 10
73
El numero mayor es: 99
```

2. Ejercicio Tabla

Para el ejercicio tres, el programa imprime una tabla de valores, primero imprime los encabezados con los números del 1 al 5, acompañados de sus respectivos resultados al multiplicarlos por 10, 100 y 1000 y alineados en columnas.

n	n*10	n*100	n*1000
1	10	100	1000
2	20	200	2000
3	30	300	3000
4	40	400	4000
5	50	500	5000

5. Conclusiones

El primer objetivo de esta práctica se cumplió ya que se logró comprender de mejor manera el funcionamiento de un programa con ayuda de nuestro LLM, se pudo comparar lo que decían los LLM's, y el alumnado pudo discernir entre lo que podría ser útil y lo que no. El segundo objetivo también fue cumplido ya que se logró entender el funcionamiento de las estructuras cíclicas en el lenguaje de java, ayudando así a la simplificación y optimización de los programas, evitando repetición de código .

Referencias

- [1] *Estructuras de control en Java*. https://dat-science.com/estructuras-de-control-en-java/#Estructura_do-while. Recuperado el 3 de septiembre de 2026.
- [2] GeeksforGeeks. "Understanding "static" in "public static void main" in Java". En: (sep. de 2018).
- [3] W3schools.com. *Java static Keyword*. https://www.w3schools.com/java/ref_keyword_static.asp. Recuperado el 3 de septiembre de 2026.
- [4] W3schools.com. *Java User Input (Scanner class)*. https://www.w3schools.com/java/java_user_input.asp. Recuperado el 3 de septiembre de 2026.